CustomPrefixDropdown Widget Documentation

Overview

`CustomPrefixDropdown` is a reusable and customizable Flutter dropdown widget. It supports a prefix widget, generic data types, custom item UI, and flexible styling options.

Features

- Generic type support (`T`)
- Optional prefix widget (icon/text)
- Custom dropdown item builder
- Custom height, padding, and border radius
- Custom dropdown icon and background color
- Fully expandable dropdown

Full Source Code

```dart
import 'package:nanduba/constants/textfontstyle.dart';
import 'package:nanduba/export.dart';

class CustomPrefixDropdown<T> extends StatelessWidget {
  final T value;
  final List<T> items;
  final ValueChanged<T?> onChanged;

  final Widget Function(T item) itemBuilder;

  final Widget? prefix;

  final double height;
  final EdgeInsets padding;
  final BorderRadius borderRadius;
  final Color borderColor;
  final Widget? icon;
  final Color? dropdownColor;

  const CustomPrefixDropdown({
    super.key,
    required this.value,
    required this.items,
```

```
  required this.onChanged,
  required this.itemBuilder,
  this.prefix,
  this.height = 50,
  this.padding = const EdgeInsets.symmetric(horizontal: 12),
  this.borderRadius = const BorderRadius.all(Radius.circular(16)),
  this.borderColor = AppColors.cBEBEBE,
  this.icon,
  this.dropdownColor,
});

@override
Widget build(BuildContext context) {
  return Container(
    height: height,
    padding: padding,
    decoration: BoxDecoration(
      border: Border.all(color: borderColor, width: 1),
      borderRadius: borderRadius,
    ),
    child: Row(
      children: [
        if (prefix != null) ...[
          prefix!,
          const SizedBox(width: 4),
        ],
        Expanded(
          child: DropdownButtonHideUnderline(
            child: DropdownButton<T>(
              style: Textfontstyle.TextStyle14w400c212121poppins.copyWith(
                fontSize: 10.sp,
                color: AppColors.textColor,
              ),
              value: value,
              isExpanded: true,
              icon: icon ?? SvgPicture.asset(
                AppSvgs.arrowdown,
                color: AppColors.darkGrey,
              ),
              dropdownColor: dropdownColor ?? AppColors.white,
              borderRadius: borderRadius,
              onChanged: onChanged,
              items: items
                  .map(
                    (item) => DropdownMenuItem<T>(
                      value: item,
                      child: itemBuilder(item),
                    ),
```

```
                    )
                    .toList(),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

Parameters

Parameter	Type	Description
`value`	`T`	Currently selected value
`items`	`List<T>`	List of dropdown values
`onChanged`	`ValueChanged<T?>`	Callback when value changes
`itemBuilder`	`Widget Function(T)`	Builds each dropdown item
`prefix`	`Widget?`	Optional widget before dropdown
`height`	`double`	Height of the dropdown
`padding`	`EdgeInsets`	Inner padding
`borderRadius`	`BorderRadius`	Corner radius
`borderColor`	`Color`	Border color
`icon`	`Widget?`	Custom dropdown icon
`dropdownColor`	`Color?`	Dropdown background color

Usage Example

```dart
// Example model
class CarBrandModel {
  final String name;
  final String logo;

  CarBrandModel({required this.name, required this.logo});
}
```

```dart
// Sample data
final List<CarBrandModel> carBrandList = [
  CarBrandModel(name: 'BMW', logo: 'bmw.png'),
  CarBrandModel(name: 'Toyota', logo: 'toyota.png'),
  CarBrandModel(name: 'Audi', logo: 'audi.png'),
];

// Selected value
late CarBrandModel selectedCarBrand = carBrandList.first;

// Widget usage
CustomPrefixDropdown<CarBrandModel>(
  value: selectedCarBrand,
  items: carBrandList,
  onChanged: (value) {
    setState(() {
      selectedCarBrand = value!;
    });
  },
  prefix: const Icon(Icons.directions_car),
  itemBuilder: (item) {
    return Text(item.name);
  },
);
```

Default Value Handling

You can initialize the dropdown with a default value using:

```dart
late CarBrandModel selectedCarBrand = carBrandList.first;
```

⚠️**Important:** Ensure `carBrandList` is not empty before using `.first`, otherwise the app will crash.

Safer Alternative

```dart
CarBrandModel? selectedCarBrand;

@override
void initState() {
  super.initState();
  if (carBrandList.isNotEmpty) {
    selectedCarBrand = carBrandList.first;
```

```
  }
}
```

Then check for null before building the dropdown.

```dart
String selectedCountry = 'USA';

CustomPrefixDropdown<String>(
  value: selectedCountry,
  items: ['USA', 'UK', 'Canada'],
  onChanged: (value) {
    setState(() {
      selectedCountry = value!;
    });
  },
  prefix: const Icon(Icons.flag),
  itemBuilder: (item) {
    return Text(item);
  },
);
```

Notes

- Ensure the selected value exists in the `items` list
- Use `itemBuilder` to fully customize dropdown items
- The `prefix` widget is optional

♍You can edit this document directly on this page and download it anytime.